kiri

Container runtimes on macOS

Kiri vs Docker Desktop vs OrbStack vs Colima · 3 November 2026 · Apple M3 Max, 64 GB · **every engine limited to 2 CPUs and 8 GB** · full run

15 / 17
tests won by Kiri

18 ms
`docker run --rm`, 8.3x faster than the next

2.5 ms
API container lifecycle, ~35x faster

242 ms
3-layer build, 2.1x faster

Results (medians)

Test	Kiri	Docker Desktop	OrbStack	Colima	Winner
`docker run --rm`	**18.3 ms**	178 ms	279 ms	152 ms	Kiri, 8.3x
`docker exec`	**13.1 ms**	47.7 ms	32.5 ms	36.8 ms	Kiri, 2.5x
API container lifecycle	**2.5 ms**	163 ms	238 ms	88 ms	Kiri, ~35x
API exec	**0.4 ms**	17.1 ms	18.3 ms	20.8 ms	Kiri, ~43x
Build (3 layers, no cache)	**242 ms**	527 ms	1048 ms	505 ms	Kiri, 2.1x
Warm pull	**350 ms**	2798 ms	1569 ms	2893 ms	Kiri, 4.5x
Parallel starts	**176 /s**	11.4 /s	8.2 /s	12.9 /s	Kiri, 13.7x
Density	**20.7 /s**	3.2 /s	2.9 /s	4.1 /s	Kiri, 5.0x
stdout	**297 MB/s**	45 MB/s	87 MB/s	63 MB/s	Kiri, 3.4x
stdin	**189 MB/s**	90 MB/s	158 MB/s	123 MB/s	Kiri, 1.2x
Bind mount write	**2042 MB/s**	1425 MB/s	1596 MB/s	1272 MB/s	Kiri, 1.3x
Bind mount read	2497 MB/s	1354 MB/s	3240 MB/s	**5235 MB/s**	Colima, 1.6x
Volume write	**1974 MB/s**	1920 MB/s	1216 MB/s	1779 MB/s	Kiri, 1.03x (tie)
Volume read	**23225 MB/s**[1]	4097 MB/s	2692 MB/s	5391 MB/s	Kiri, 4.3x
iperf3, single stream	**26.2 Gbit/s**[2]	1.1 Gbit/s	24.0 Gbit/s	2.4 Gbit/s	Kiri, 1.09x
iperf3, 4 streams	13.7 Gbit/s[2]	1.6 Gbit/s	**17.7 Gbit/s**	2.5 Gbit/s	OrbStack, 1.3x
New HTTP connections	**2741 req/s**[2]	865 req/s	2389 req/s	1484 req/s	Kiri, 1.15x

Lower is better for times, higher for rates; the winner is compared with the runner-up. Kiri 15 wins, OrbStack 1, Colima 1, Docker Desktop 0. [1] Served from the page cache; an upper bound, not disk speed. [2] From a second network-only run (see How we tested, item 6).

Tail latency (p95)

Test	Kiri	Docker Desktop	OrbStack	Colima
`docker run --rm`	22.5 ms	278 ms	443 ms	165 ms
`docker exec`	15.7 ms	64.2 ms	37.7 ms	40.9 ms
API lifecycle	3.2 ms	266 ms	284 ms	103 ms
API exec	0.6 ms	20.6 ms	26.7 ms	26.2 ms
Warm pull	366 ms	2891 ms	1611 ms	2926 ms

Build	257 ms	653 ms	1080 ms	519 ms

Verdict

At equal compute, Kiri is the fastest runtime on this Mac in 15 of 17 tests: start, exec, direct API, build, pull, parallel starts, density, stdio, writes, volume reads, single-stream network throughput and new HTTP connections.
OrbStack still has more throughput with 4 parallel streams, and Colima reads bind mounts fastest; Docker Desktop wins no test. Yesterday's two Kiri failures (build and iperf3) are fixed. These are measurements on one machine, not universal claims.

How we tested

1. **One machine.** Apple M3 Max, 16 cores, 64 GB, macOS 27.0.1, mains power. Kiri was measured on 3 November 2026 (04:57–05:00 UTC, network suite again 05:01–05:03, host load about 4–5). Docker Desktop, OrbStack and Colima are the 2 November run (18:20–18:34 UTC, load about 7–9), with the same settings and the same harness.
2. **Same compute: 2 CPUs, 8 GB each**, confirmed with `docker info` before each run.

Engine	Setting	docker info
Kiri (development build with the layer fix)	`vm.json`: 2 CPUs, 8192 MiB	2 CPUs, 7.8 GiB
Docker Desktop 4.90.0	Settings → Resources: CPU 2, Memory 8 GB	2 CPUs, 7.7 GiB
OrbStack 2.2.4	`orb config set cpu 2`, `memory_mib 8192`, restart	2 CPUs, 7.8 GiB
Colima 0.10.3	`colima start --cpu 2 --memory 8 --vm-type vz --mount-type virtiofs`	2 CPUs, 7.7 GiB

3. **Nothing else running in the engines.** Kiri ran as a fresh engine of its own with no other containers.
4. **Full run, suite defaults:** 30 latency samples after 3 warmups, 5 I/O samples, 200 MiB stdio transfers, 1 GiB `fsync` storage writes, 5-second network streams (4 parallel), 200 new HTTP connections, 20 containers for density.
   `./kiri-bench run --engine kiri --command "docker --context kiri-layers"`
5. **What changed in Kiri since 2 November.** The empty-file bug is fixed: the VM was stopped before the guest had flushed its disk, so recently unpacked image layers could come back with every file empty. Kiri now waits for the guest to power off, flushes each layer before using it, and re-unpacks any layer that was cut short. Build and iperf3 now pass.
6. **One open Kiri issue.** In the full run, all five 4-stream iperf3 runs failed at once ("control socket has closed") while Kiri was relaying published ports over its fallback path. A second run of the network suite minutes later passed every test; its iperf3 and HTTP numbers are the ones shown.

Full method and the reproduction guide: github.com/libracoder/kiri-bench → docs/benchmarks/2026-11-03. Rerun it on your own hardware before deciding.